

Can a Language Model Score Credit as Well as Machine Learning?

Classical Machine Learning versus Large Language Models for Credit Scoring: Offline Simulation and Explainability

Alessandro Mezzanotte

Jad Zoghaib

Francesc Cañavate Quero

Project Supervisor - Josep Puig Sallés

Agenda

Four parts, with the spotlight on the language-model approach.

1

Introduction

The question we set out to answer, and the project at a glance.

2

Data & the ML baseline

The loan dataset and a quick read on the classic model's score.

3

The LLM approach

Our main focus: experiments, costs, trust, and reasoning.

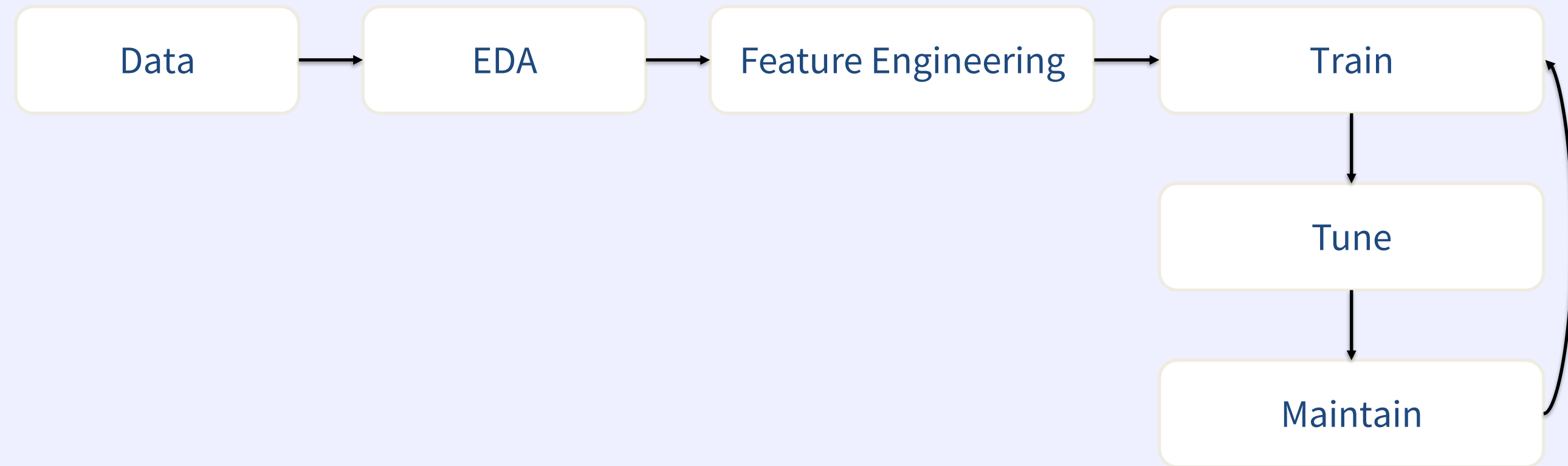
4

Conclusion

When to use which, the architecture we'd recommend, and the limits.

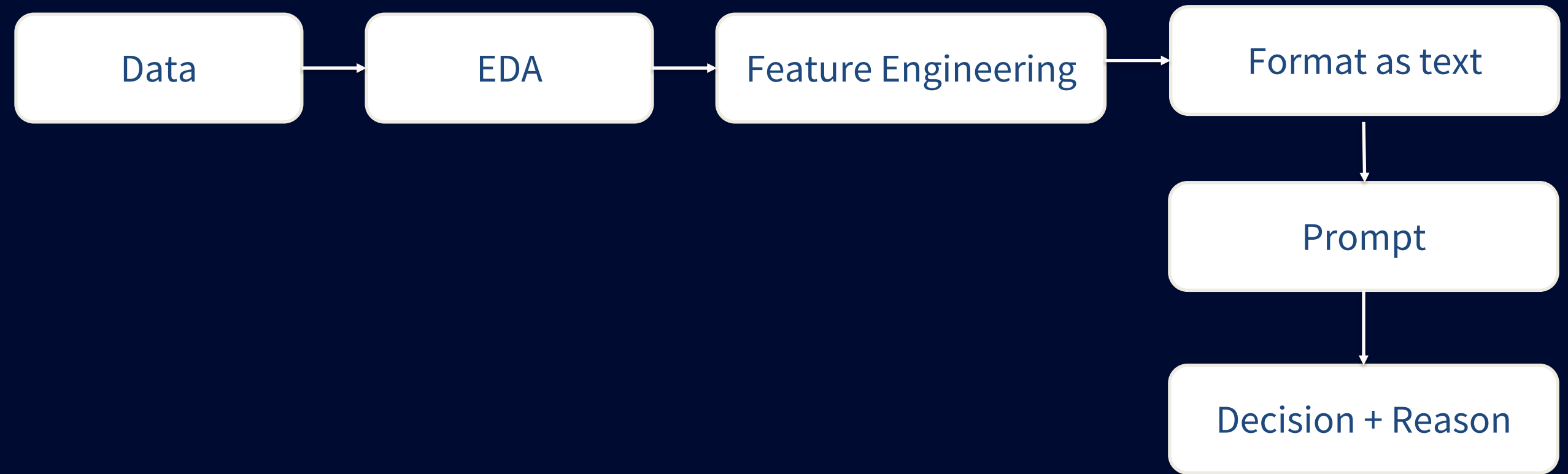
Machine Learning

- Needs labelled data
- Weeks of engineering
- Retrain to adapt to changing data



Large Language Model LLM

- No task-specific training
- No train/tune/maintain loop
- Decision comes with reasoning and explanation



Research Question

Can a prompt replace the machine learning pipeline?

And if so, when is the LLM the right tool for credit scoring, and when is ML still the better call?

02

Our Data and the ML Model Training

We cleaned the data, engineered the features, split it into train and test sets, and trained the ML model.

The Data: peer-to-peer loans from LendingClub

LendingClub loans (2012–2014 subset)

For each loan we know the borrower's profile and whether it was repaid

Available Features

Borrower

income · job length · home ownership · FICO score · debt-to-income

The Loan

amount · interest rate · term · monthly payment · grade · purpose

Credit History

open accounts · past delinquencies · recent inquiries · bankruptcies

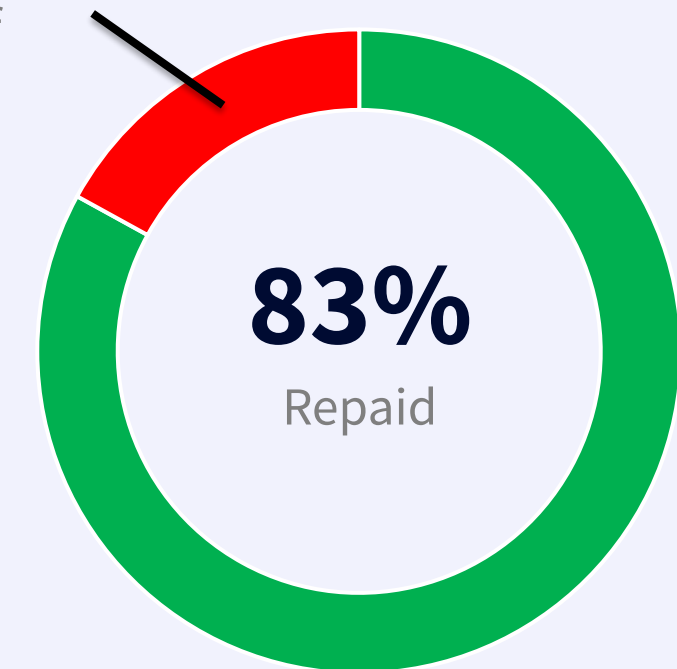
Repayment Rate

17%

CO - Charged Off
(default)

83%

Repaid



Around 30 facts per loan; the same set goes to both models

Finding the baseline ML model

The full machine learning pipeline, and what we did in each step



Clean

fix gaps, encode fields

Kept only decided loans
(fully paid / charged off)

Used **only 2012-2014** subset:
stable macroeconomic period
and loans had a text feature we
tested

Turned words into numbers



Engineer features

30 features per loan

Added strong risk signals: FICO
band, recent late payments,
recent credit inquiries, length of
credit history

Removed post-loan facts; no
data leakage

Exact same features are used for
LLM



Tune

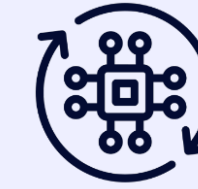
search best settings

Trained: Logistic regression,
XGBoost, small neural network

Rebalanced for the minority
class (17% defaults)

Decision cut-off for catching
defaults

XGBoost as the ML model compared against the LLM



Gradient boosting (XGBoost), the standard for tables

69%

Accuracy

Overall

Right call on about 7 of every 10 loans

59%

Recall

Defaults

Of the loans that truly default, it catches around 6 every 10

29%

Precision

Defaults

When it flags a loan as risky, it's right ~ 3 in 10 times. The rest are false alarms

0.39

F1 Score

Defaults

The balance between catching defaults and not over-flagging

0.71

AUC

Ranking

Ranks a bad loan above a good one 71% of the time

A model that blindly approves everyone would score 83% accuracy and catch zero defaults.

Ours is deliberately tuned to flag the bad loans, **trading a little overall accuracy for far better default detection.**

These metrics are not the ideal scores for a ML model, and the reason is probably structural of the dataset.

The features are not predictive enough, and don't help the models

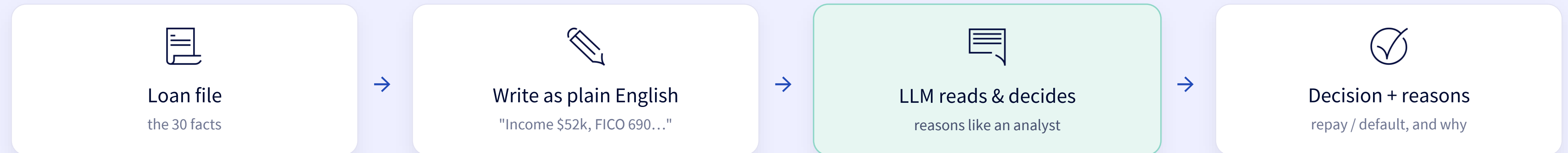
03

The LLM approach

The heart of the project. Can we get a language model to make good credit calls, and can we trust it enough to use it?

How a language model scores a loan

No retraining. We hand it the loan as plain text similar to briefing a junior analyst.



You are a credit risk analyst. Given a loan application's features, predict whether the borrower will fully repay the loan or default (charge off).

Respond ONLY with valid JSON in this exact format:

```
{"prediction": <1 or 0>, "reasoning": "<brief explanation>"}
```

Predict the outcome for this loan application:

- Loan amount requested (\$): 10,000
- Revolving utilization rate (%): 92.7
- Loan term: 36 months

```
{
  "prediction": 0, (Default)
  "reasoning": "This application shows several elevated risk factors,
including low income relative to the loan size, very high revolving
utilization, a prior bankruptcy/public record, and only short
employment history. Although the borrower has a moderate FICO score
and no recent delinquencies, the overall profile suggests higher
default risk."
}
```

Our Experiment Plan: Five Levers

We didn't just adjust the phrasing; we refined and validated the entire framework surrounding the model.

1

Pick the model

5 models tried · winner: GPT-5.4

GPT-5.4 ✓

Gemini 2.5 Pro

Gemini 3.5 Flash

Claude Sonnet 4.6

Claude Opus 4.8

2

Design the prompt

7 ways of asking
Winner: the base one

Base ✓

Chain of thought

Few shot

4 factor

Risk guide

Top features

Conservative

3

Make it cheaper

GPT-5.4 · winner: prompt caching

Prompt caching ✓

Compacting (TOON)

Batching

→ kept caching: ~90% cheaper, no accuracy loss

4

Blend with ML

mix the two models together

Soft blend

Confidence gate

Intersection

Risk scorers

→ GPT 5.4 makes fewer errors, but they're the same the ML model makes

5

Give it memory (RAG)

show it similar past loans as evidence

Semantic IDs (FLARKO) ✓

Nearest neighbours

Hybrid of both

→ the lever we build on

6

Final test

one clean run on untouched loans

Sealed 1,000-loan benchmark, with euro impact

BEFORE LEVER 1

How we setup the batches

- *

Full Dataset
Lending Club 2012-2014 loans
- 1

Tuning Sample
100 loans
- 2

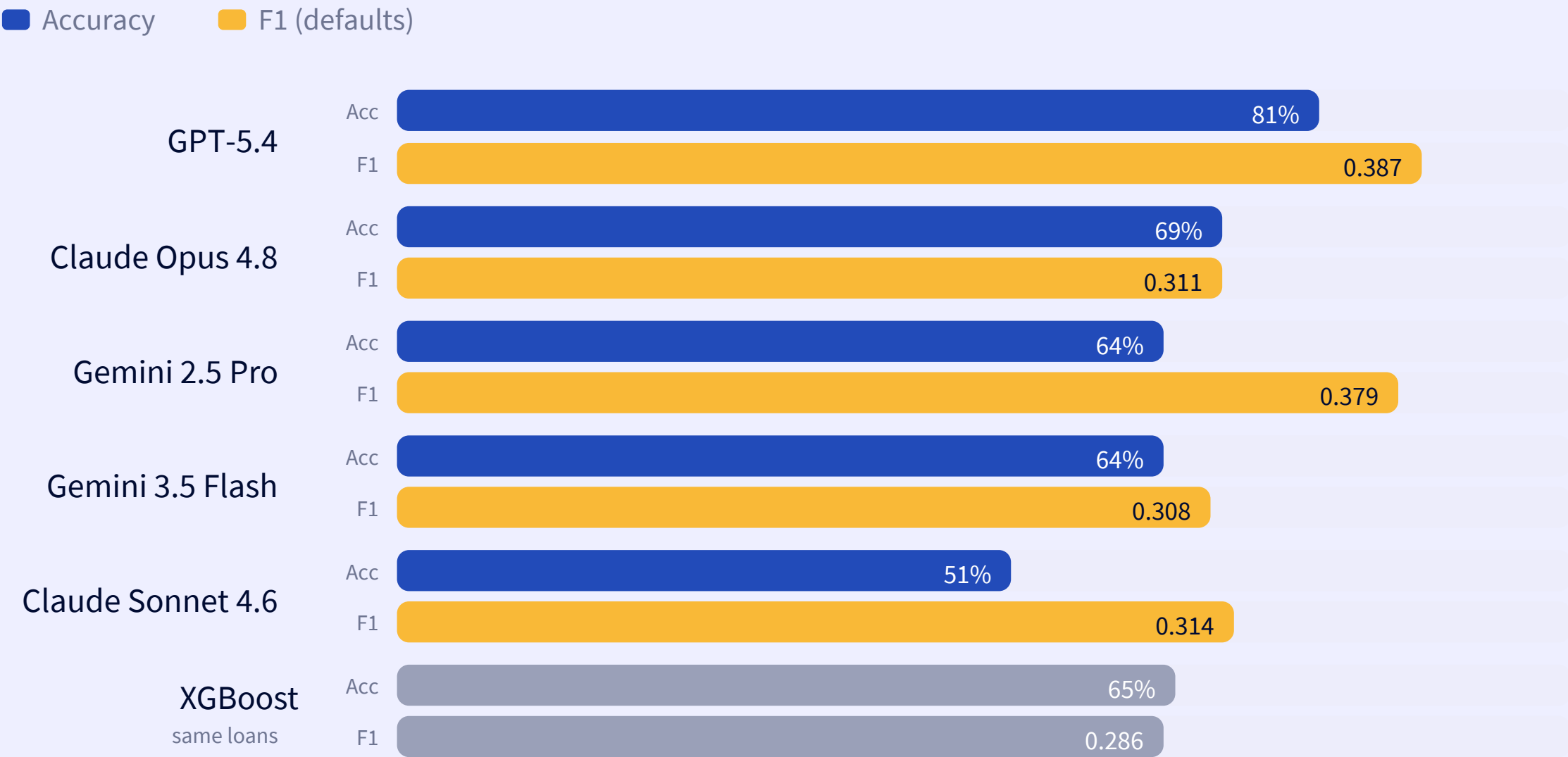
Robustness Batch
100 more loans
- 🔒

Sealed Test Batch
1000 loans, locked

Each batch is a **fresh, non-overlapping** draw. The sealed batch is opened exactly once, at the very end.

Which model wins, and is it reliable?

Same 100 loans, best setting for each. Then re-run the winner to more times to check for consistency.



Winner

GPT-5.4

The strongest model to beat the ML baseline on both accuracy and catching defaults. Fast and cheap to run well

95%

of the time a loan gets the same answer on a re-run

~2%

accuracy wobble between repeat runs

✓ GPT-5.4 does better without the free-text borrower notes, so we use the clean, numbers-only setting. Consistency is a big plus for real use.

Seven ways of asking

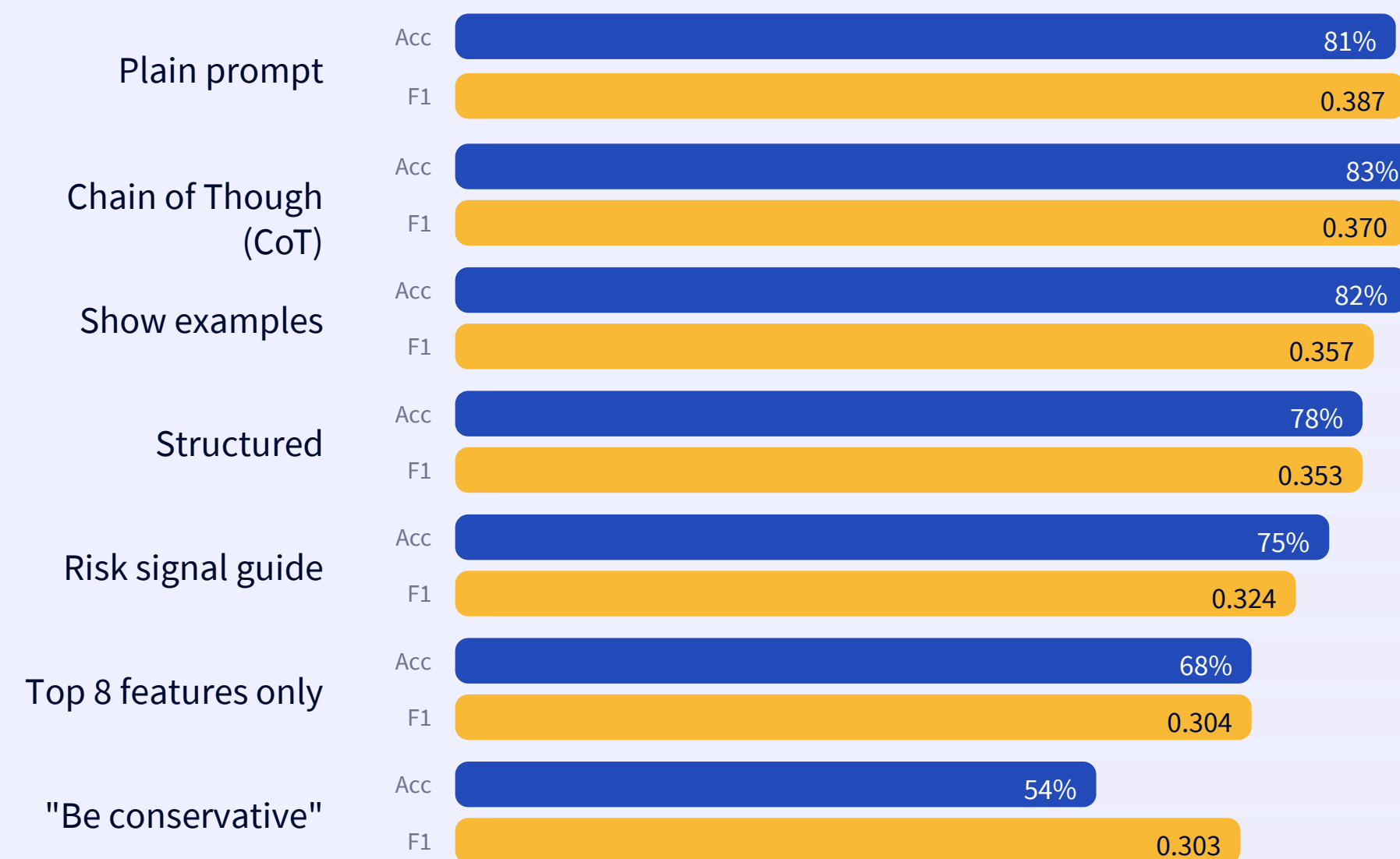
Default-catching score, single run. Some looked promising until we tested them twice.

Base prompt (control)	"You are a credit risk analyst. Given a loan application's features, predict whether the borrower will fully repay the loan or default..."
Conservative	System persona: "You are a risk-averse underwriter... when evidence is mixed or uncertain, err on caution and predict Charged Off."
Chain of Thought	"Before predicting, reason step by step: identify risk signals, weigh protective factors, then conclude." + user asks "Analyze this loan step by step..."
Show examples	User prepends 4 solved past loans before the question: "Here are examples of past outcomes... now predict this new one."
Top features only	Feeds just the 8 most important features (from XGBoost), not all 30: "... (key features only)."
Structured	System hands it a checklist: "Evaluate using: 1) income vs. installment, 2) debt burden (DTI, utilization), 3) credit history, 4) loan characteristics."
Risk-signal guide	System injects an expert rulebook with thresholds: "Grades E–G default >30%; interest >18% high-risk; DTI >35% strong signal; utilization >80%..."

Seven ways of asking

Default-catching score, single run. Some looked promising until we tested them twice.

■ Accuracy ■ F1 (catching defaults)



The key takeaway

The fancier the prompt, the more it gets biased.

GPT-5.4 already seems fairly balanced on this task, so adding more structure mostly gets in the way.

- The best-performing prompts are separated by only small margins, and once we repeated the test, those gains did not hold consistently.
- That suggests the differences at the top are more likely noise than a real improvement from prompt engineering.



No clever prompt reliably beat the base one. The simplest ask is the winner

Three ways we tried to make it cheaper

Each call costs tokens. We tested three ways to spend fewer and watched what it did to accuracy.



Compact Format (TOON)

Pack the loan into a tight table instead of full sentences. About 18% cheaper, with almost no accuracy loss (81% → 78%).



Batching (many loans in one call)

Cheaper still (30–60%), but accuracy collapses to 62%. The model gets jumpy and rejects good borrowers — a false economy.



Prompt Caching

Winner

Reuse the repeated instructions instead of resending them. Up to a ~90% discount (on the repeated prefix) with zero accuracy cost.

ACCURACY BY METHOD



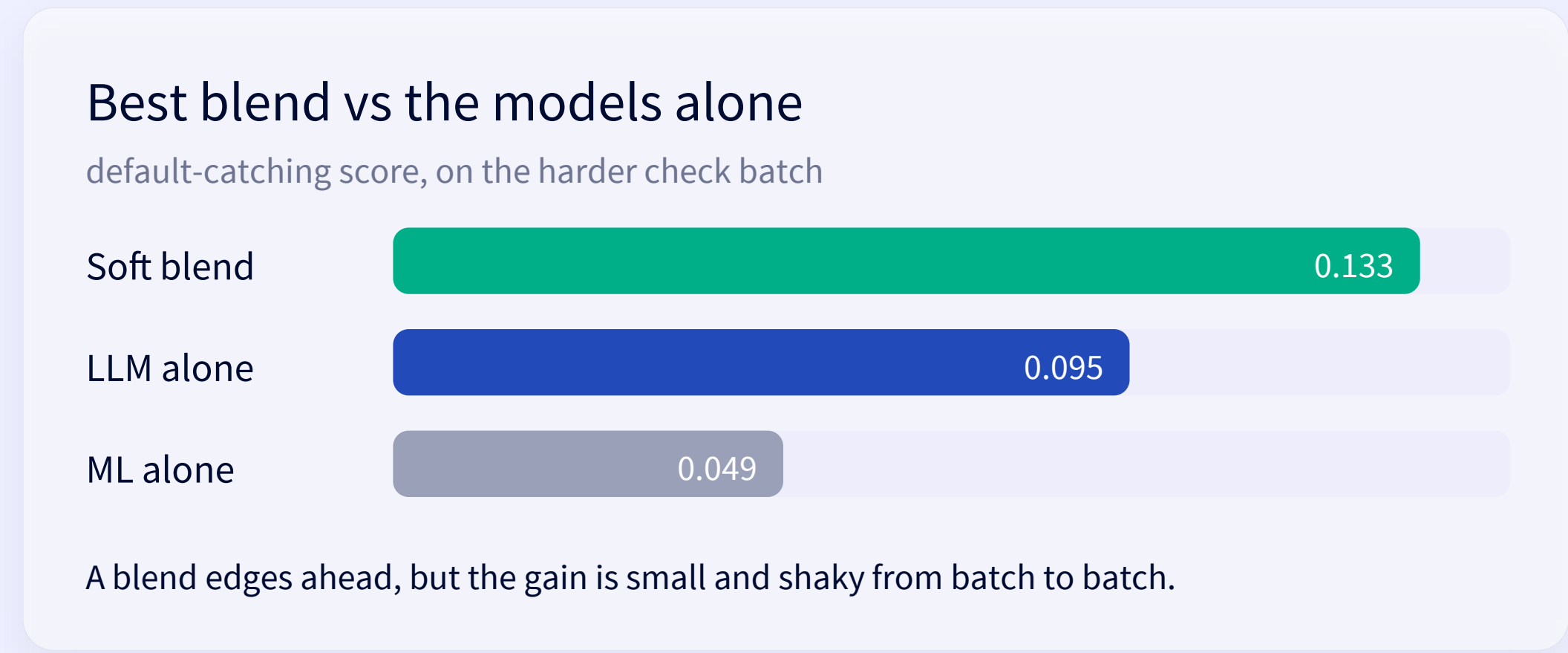
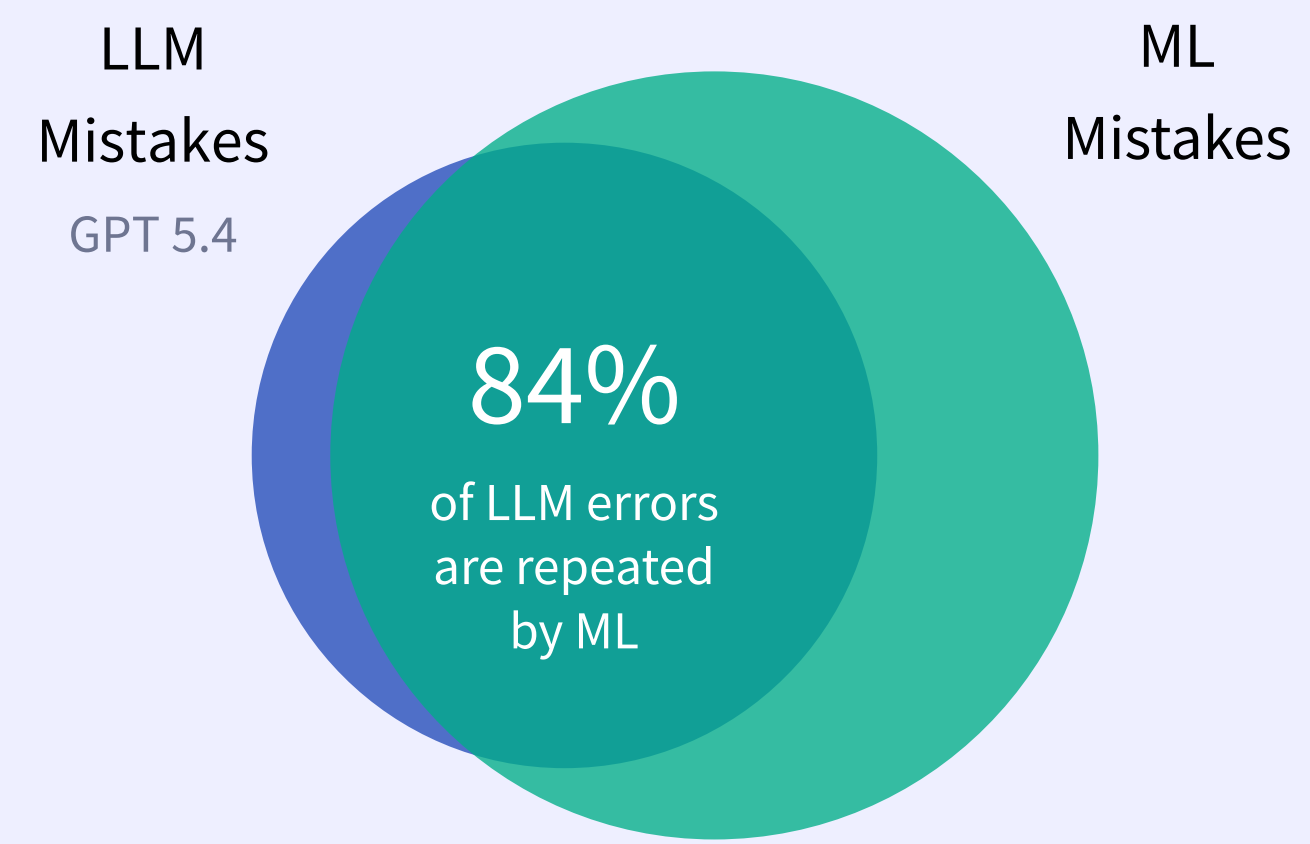
Caching keeps the best accuracy and cuts the bill, so it beats the “squeezing” approaches



With OpenAI prompt caching is automatic when you send repeated instructions and examples, cutting input costs

GPT 5.4 errs less, and its errors nest inside XGBoost

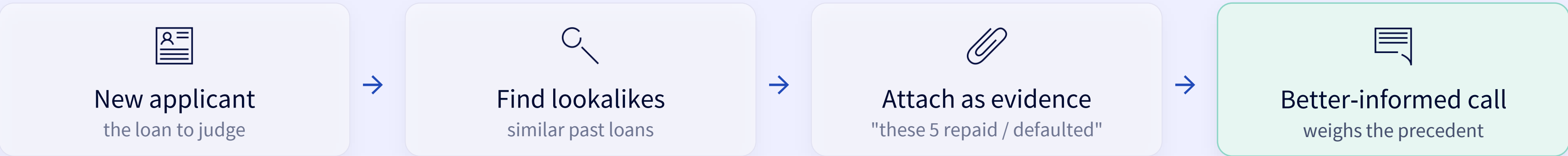
Tempting idea: let each cover the other's blind spots. We tested several ways to blend them



When the LLM is wrong, XGBoost is usually wrong too, so it adds almost nothing
The small blend gain is shaky batch to batch, not a compelling direction over the LLM alone

Showing it precedent: building a RAG system

Instead of judging in a vacuum, we pull up similar past loans with known outcomes and hand them over as evidence. We did this in three ways.



A · Semantic IDs (FLARKO)

Give every loan a short "family code" so lookalikes share it. Match the family, then do a two-step borrower-then-loan filter.

B · Nearest neighbours

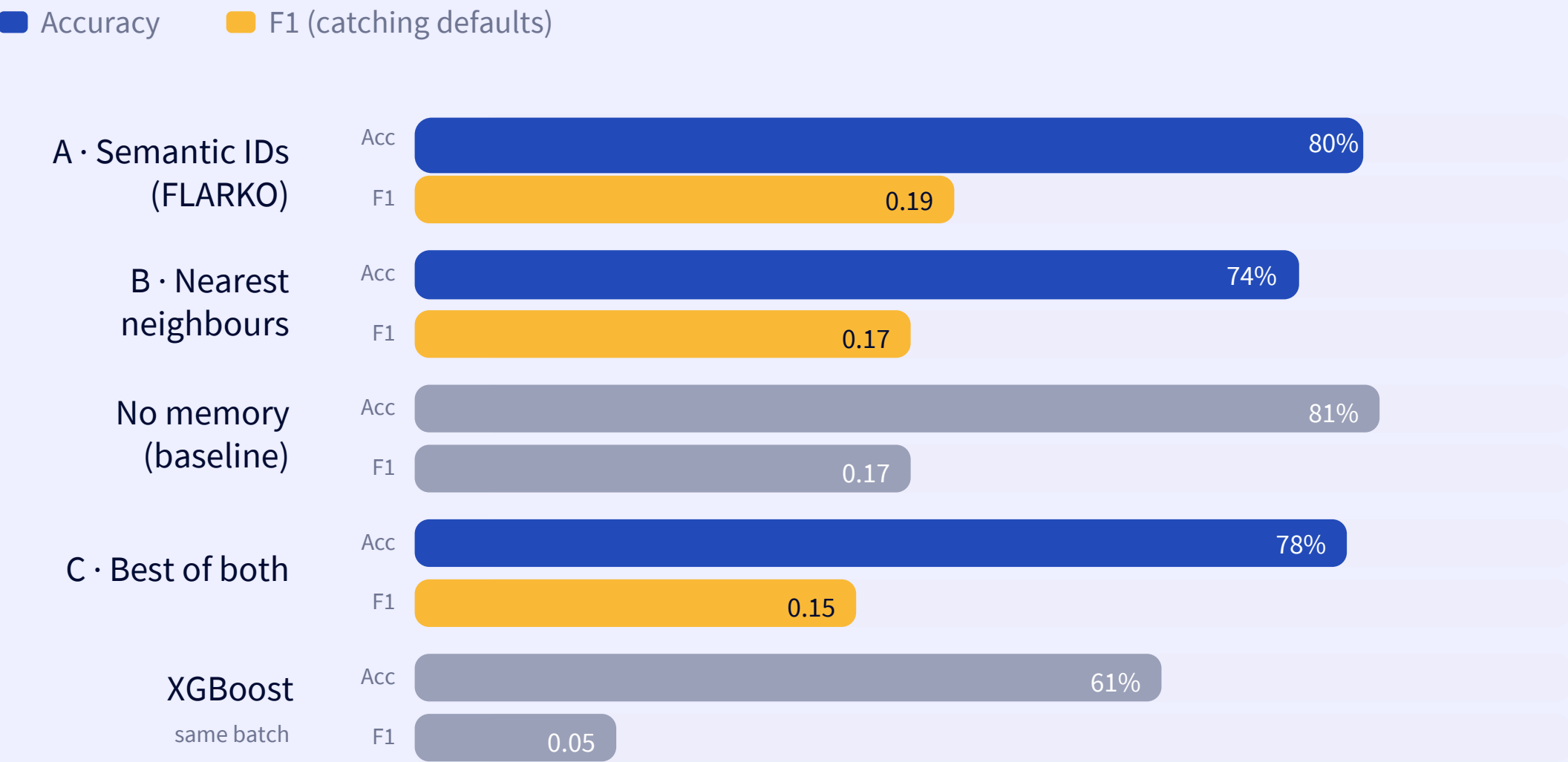
Simply scan the whole history and pick the most similar past loans overall.

C · Best of both

Merge the picks from A and B into one combined shortlist of precedents.

Did the memory help?

All three vs the same model with no memory, on our validation batch.



Overlap check

Why "best of both" came last

We measured how often methods A and B pick the same precedents. They overlap a lot, so merging them adds little and can dilute the stronger one. The smart fingerprint alone is best.



Memory nudges the right way on a tiny batch. More importantly, the setup, not the wording, is the real research lever, and that's what we build on next.

The winning recipe

Pulling the experiments together and running them on a final 1000 loans held-out test set

Best model

GPT-5.4

strongest call-maker, fast and cheap to run well

Best prompt

Base prompt

simple one that simply gives the task of making the prediction; no notes, no clever wording beat it

Best aid

Semantic IDs (FLARKO)

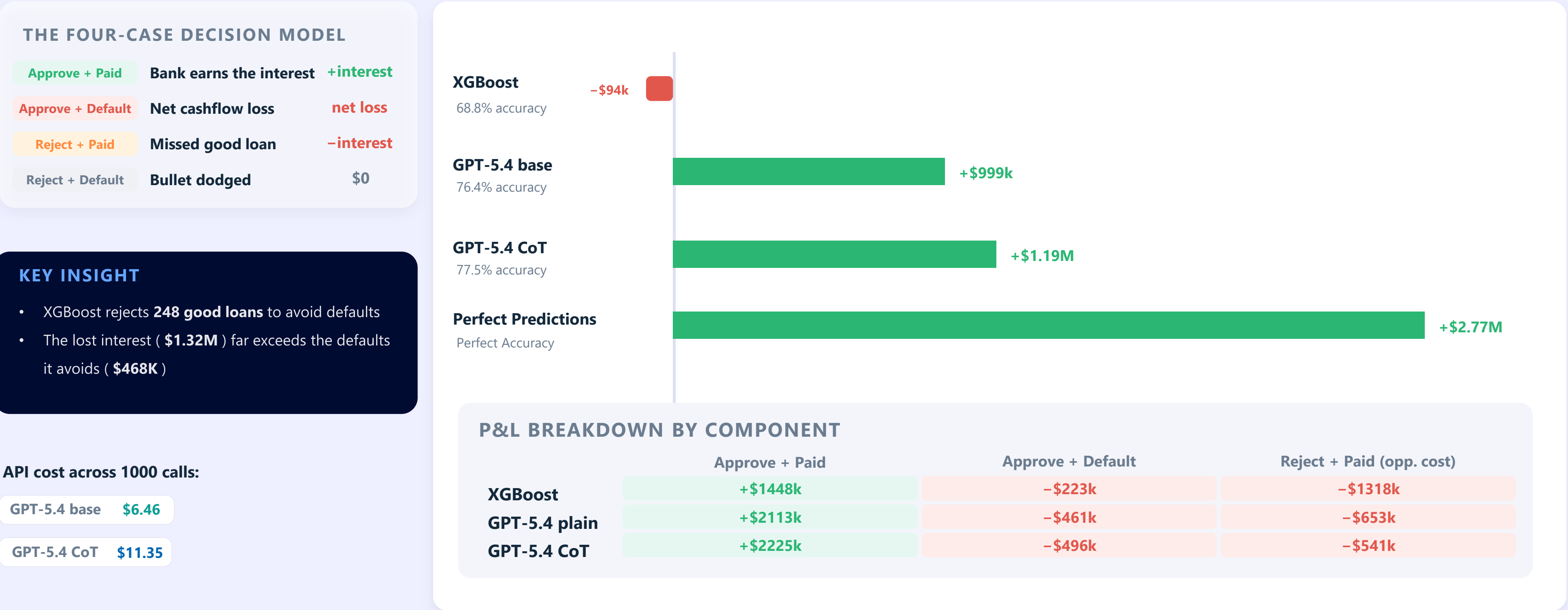
Used for explainability, not accuracy — retrieved precedents help analysts audit a decision from an otherwise black-box LLM



On the fair final test, GPT-5.4 beats the classic model on accuracy and on Profit & Loss. XGBoost comes ahead on catching defaults

What would each approach actually cost the bank?

Four-case per-loan payoff on 1000 untouched test loans, actual LendingClub cashflows, not assumptions.



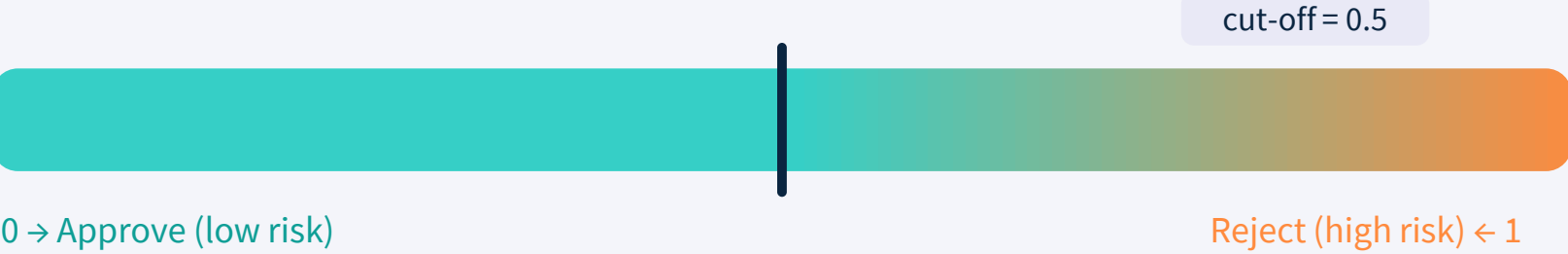
How each model takes the decision

XGBoost gives you a probability to cut at any threshold
The LLM sounds equally certain whether right or wrong, making threshold-setting impossible.



XGBoost: a probability you can act on
Outputs a number 0 to 1. You set the cut-off. Above it: reject. Below: approve.

RISK PROBABILITY OUTPUT PER LOAN



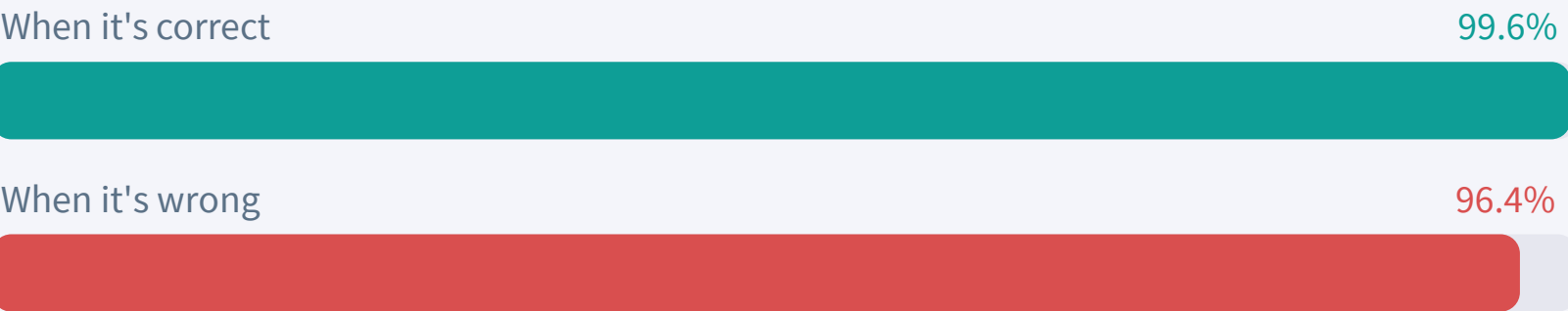
You can move the cut-off to favour precision or recall depending on the bank’s needs.

FEATURE IMPORTANCE



LLM: near-certainty, right or wrong
Confidence extracted from token logprobs. Almost no gap between correct and incorrect calls.

HOW CONFIDENT IT SOUNDS (LOGPROB-DERIVED)



3.2% gap — the model sounds almost as certain when it is wrong. No meaningful cut-off can be set.

WHAT A GPT-5.4 JUDGE FINDS IT CITES

- 1 FICO band
- 2 Recent delinquencies
- 3 Bankruptcies and public records
- 4 Card use, DTI, grade, rate

AUC 0.62 — weaker risk ranking

XGBoost = better risk meter (AUC 0.706 — its probability score is meaningful for threshold tuning).

LLM = better decision-maker (76% accuracy) but its confidence is too high to be informative

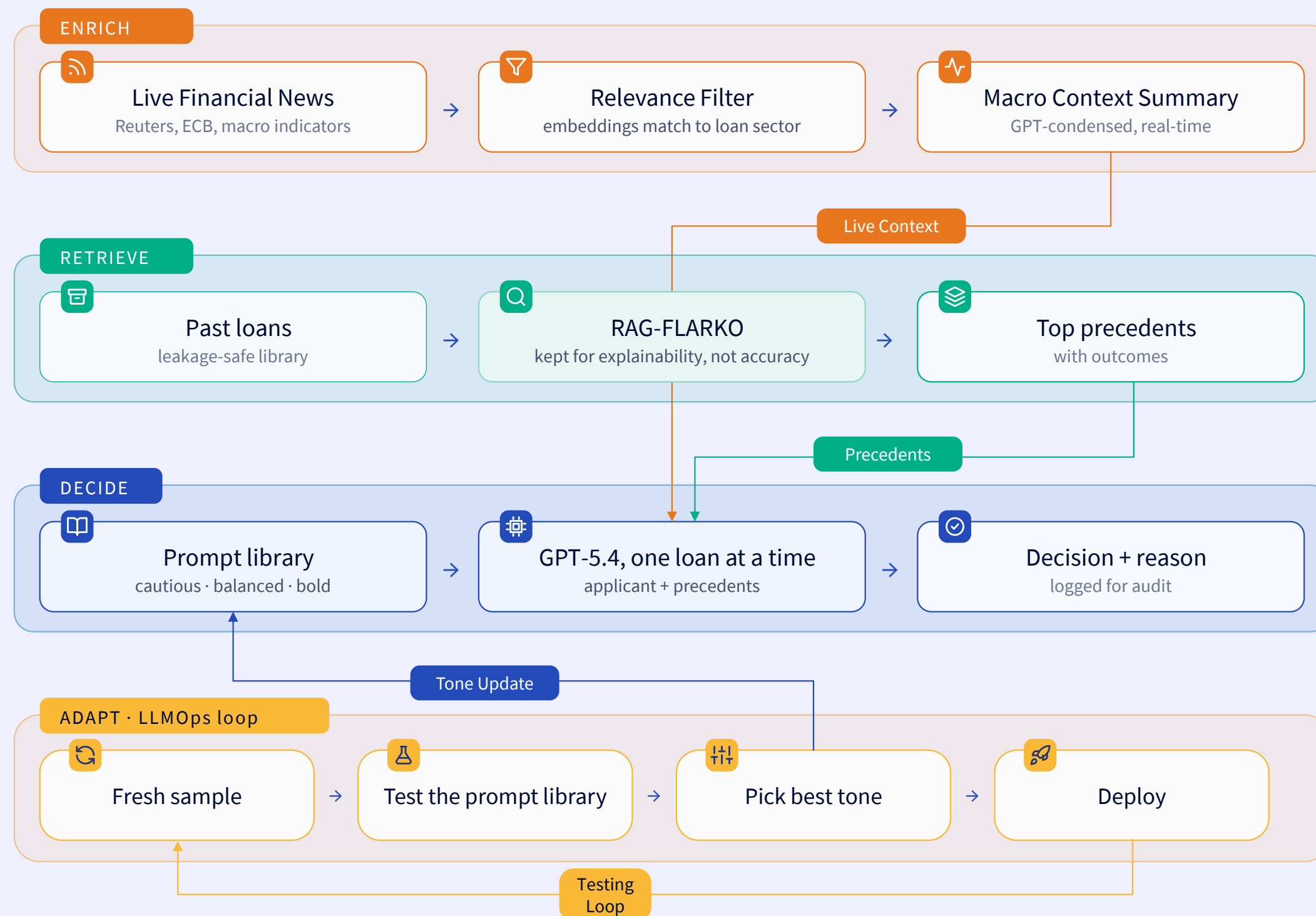
04

Conclusion & Recommendation

When to use which, the architecture we'd recommend if we built it, and the limits to our approach

If we built it: the recommended architecture

Lock the best setup (memory + individual scoring) and run it inside a living loop that re-tunes the tone over time.



STEPS

- 1 Pull live financial news and filter for relevance to the loan's sector
- 2 Build a library of past loans as precedents
- 3 For each applicant, run RAG-FLARKO to find the closest historical precedents with their outcomes to facilitate analyst work.
- 4 Score one applicant at a time with GPT-5.4, injecting three inputs simultaneously
- 5 Log every decision + written reason for audit and regulatory traceability.
- 6 Periodically test the prompt library (cautious · balanced · bold) & pick the tone that best fits current economic conditions.
- 7 Deploy the winning tone and loop
- 8 Keep a human in the loop at the decision gate

When is the LLM the better choice?

It's not "LLM wins" everywhere, it's about fit.



Reach for the LLM when...

- Data is small or scarce
- The ML model isn't robust enough
- You need to move fast, with no months of engineering
- You want a plain-English reason with every call



Stick with classic ML when...

- You have lots of clean, labelled data
- You need a well-calibrated risk score
- You score at huge volume at the lowest cost
- The problem is stable and well understood

“ Our case, small samples and a modest ML model, is exactly where the LLM shines. But the real deliverable is the method, not this one model.

Doing it responsibly

A credit decision affects real people, so the guard rails matter as much as the score, and so do the limits we'll name out loud.

DOING IT RESPONSIBLY

GDPR & the right to an explanation

Every decision comes with a plain-English reason a more natural fit for "explain why I was declined".

Human in the loop

A decision-support tool, not an autopilot. Final calls stay with people.

Fairness watch

We already caught one unfairness (batching).
The same borrower must get the same decision regardless of context.

KEY PROJECT LIMITATIONS

Small experimentation batches

Our hardest batch has only ~9 defaults, so default-catch scores swing a lot. We lean on accuracy and multi-run averages.

No external news yet

Adding financial news to the memory would risk leaking the future and biasing results today. A strong candidate for a later version.

Was the data in the model's training?

LendingClub is public, so the LLM may have seen it. Open question: a head start, a confound, or fine for a prototype?

Greedy search experimentation

We froze the winning model before testing prompts, formats and memory, so we might have missed a rival model that might have excelled under those conditions. This design was driven due to budget restrictions



Thanks for the Attention

Q&A